PlainML

Compiler Construction Project README

A domain-specific language for fixed-weight feed-forward neural network inference

Course	CS-4031 Compiler Construction
Semester	Fall 2026
Platform	Linux / Ubuntu / WSL
Implementation	Flex + Bison + C + LLVM IR

Repository: https://github.com/fastcel/PlainML

1. Project Overview

PlainML is a small domain-specific language (DSL) designed to describe and execute fixed-weight, fully-connected feed-forward neural networks. The project demonstrates the complete compiler pipeline required in the course: lexical analysis with Flex, parsing with Bison, semantic validation, direct interpretation in C, LLVM IR generation, and ahead-of-time native compilation.

The project intentionally focuses on compiler construction rather than machine-learning training. Network weights and biases are written directly in the PlainML source program and are treated as fixed constants during prediction.

1.1 Supported Language Scope

- Named scalar network inputs.
- One or more sequential layers.
- One or more neurons per layer.
- Weighted connections and per-neuron bias values.
- ReLU and step activation functions.
- A predict statement containing concrete input values.
- Feed-forward inference only; later layers may reference neurons from the immediately previous layer.

Not included: training, backpropagation, datasets, tensors, GPUs, convolutional/recurrent networks, transformers, or external machine-learning frameworks.

2. Compiler Architecture

PlainML uses the same front end for both project phases. Flex converts characters into tokens, Bison validates the grammar and executes C semantic actions, and the parsed network is stored in C structures representing inputs, layers, neurons, connections, weights, biases, and activation functions.

Stage	Component	Purpose
1	Flex (lexer.l)	Recognizes keywords, identifiers, numbers, punctuation, and whitespace.
2	Bison (parser.y)	Parses PlainML syntax and invokes C semantic actions.
3	Semantic checks	Rejects invalid network references, missing values, duplicates, and wrong network usage.
4A	Phase 1 interpreter	Executes neuron arithmetic directly in C.
4B	Phase 2 AOT compiler	Emits LLVM IR for the parsed network and prediction values.
5	Clang / LLVM	Compiles output.ll into the native executable plainml_aot.

Pipeline summary: PlainML source -> Flex tokens -> Bison grammar/actions -> validated C network model -> interpreter execution (Phase 1) or LLVM IR generation (Phase 2) -> native executable.

3. Dependencies and Environment

Linux compatibility is required. The project was developed and tested in Ubuntu under WSL, and the same commands work on a normal Ubuntu/Linux installation.

3.1 Required Tools

Dependency	Role	Tested Version
Flex	Lexical analyzer generation	2.6.4
Bison	Parser generation	3.8.2
GCC	Builds the PlainML front end	15.2.0
GNU Make	Automates builds and tests	4.4.1
Clang / LLVM	Compiles generated LLVM IR	21.1.8

The exact tested versions are not hardcoded requirements; compatible Linux versions of these tools should also work.

3.2 Ubuntu Installation

```
sudo apt update
sudo apt install flex bison gcc make clang llvm libfl-dev
```

4. PlainML Language Syntax

A program contains one network definition followed by one predict statement. The first layer references declared network inputs. Every later layer references neurons from the immediately previous layer.

```
network student_success {
    inputs {
        attendance;
        study_hours;
        assignments;
    }

    layer hidden activation relu {
        neuron effort {
            attendance weight 0.30;
            study_hours weight 0.80;
            assignments weight 0.60;
            bias -0.20;
        }
    }

    layer output activation step {
        neuron success {
            effort weight 1.00;
            bias -0.50;
        }
    }
}

predict student_success with {
    attendance = 0.82;
    study_hours = 0.60;
    assignments = 0.80;
}
```

4.1 Activation Semantics

- ReLU: relu(x) = x when x > 0; otherwise 0.
- Step: step(x) = 1 when x >= 0; otherwise 0.

For each neuron, PlainML computes the weighted sum: bias + sum(source_value * weight), then applies the activation function assigned to that layer.

5. Phase 1 - Interpreter

The interpreter implementation is preserved on the interpreter-dev branch. It parses the PlainML source and executes the network directly using C code inside the Bison implementation. No intermediate code is used in this phase.

5.1 Build and Run

```
git checkout interpreter-dev
make clean
make
./plainml < examples/test.plain
```

Expected final output for the standard example:

```
Prediction:
  success = 1.0000
```

6. Phase 2 - Ahead-of-Time LLVM Compiler

The final AOT implementation is on the aot-compiler-dev branch. The parser validates the program and generates LLVM IR in output.ll. Clang then compiles that IR into a native executable named plainml_aot.

6.1 Build the Compiler Front End

```
git checkout aot-compiler-dev
make clean
make
```

6.2 Compile a PlainML Program AOT

```
make aot SOURCE=examples/test.plain
```

This command performs the following steps automatically:

1. Builds the Flex/Bison front end if needed.
2. Parses and validates the selected PlainML source file.
3. Generates LLVM IR in output.ll.
4. Runs Clang on output.ll.
5. Creates the native executable plainml_aot.

6.3 Run the Native Executable

```
make run-aot
```

Expected output:

```
Prediction:
  success = 1.000000
```

A Clang target-triple override warning may appear while compiling output.ll on some systems. In the tested Linux environment, this warning does not prevent executable generation or correct execution.

7. Testing

The AOT branch contains automated positive and negative tests. Run the complete suite with:

```
make clean
make test
```

7.1 Valid AOT Regression Programs

File	What It Tests	Expected Result
examples/test.plain	Basic two-layer network	success = 1.000000
examples/multi_neuron.plain	Multiple neurons/connections	success = 1.000000
examples/deep_network.plain	Three sequential layers	result = 1.000000

7.2 Semantic Error Tests

Run only the invalid-program suite with:

```
make test-errors
```

File	Expected Rejection
unknown_input.plain	Connection source is not a declared input.
missing_prediction_input.plain	A declared input has no prediction value.
duplicate_prediction_input.plain	A prediction input is assigned more than once.
duplicate_input_decl.plain	An input name is declared more than once.
duplicate_layer.plain	A layer name is declared more than once.
duplicate_neuron.plain	A neuron name is repeated within the same layer.

8. Semantic Validation

The final compiler performs semantic checks beyond the Bison grammar. Invalid programs terminate with a clear semantic error instead of producing incorrect code.

- First-layer connection sources must be declared inputs.
- Later-layer connection sources must be neurons in the immediately previous layer.
- The network name used by predict must match the defined network.
- Every declared input must receive a prediction value.
- A prediction input cannot be assigned more than once.
- Input names cannot be declared more than once.
- Layer names cannot be declared more than once.
- Neuron names cannot be duplicated inside the same layer.
- Maximum structure sizes are checked before inserting inputs, layers, neurons, or connections.

9. Project Files

Path	Purpose
`.gitignore`	Ignores generated lexer/parser files, binaries, LLVM IR, object files, and editor artifacts.
`Makefile`	Builds the front end, compiles PlainML programs AOT, runs executables, and executes regression/error tests.
`lexer.l`	Flex lexical specification. Recognizes PlainML keywords, identifiers, signed numeric literals, punctuation, and whitespace.
`parser.y`	Bison grammar plus C structures, semantic actions, validation, interpreter support, and LLVM IR generation.
`README.md`	Repository landing-page title.
`README.pdf`	Submission documentation for setup, building, usage, examples, tests, dependencies, and project-file descriptions.
`examples/test.plain`	Canonical two-layer student-success network.
`examples/multi_neuron.plain`	Regression example containing multiple hidden neurons and connections.
`examples/deep_network.plain`	Regression example containing more than two layers.
`examples/errors/*.plain`	Six invalid programs used to test semantic rejection.

9.1 Generated Build Artifacts

The following files are generated automatically and are intentionally ignored by Git:

- parser.tab.c and parser.tab.h - generated by Bison.
- lex.yy.c - generated by Flex.
- plainml - compiled Flex/Bison front-end executable.
- output.ll - LLVM IR produced from the selected PlainML program.
- plainml_aot - native executable compiled from output.ll.

10. Makefile Command Reference

Command	Description
`make`	Build plainml.
`make aot SOURCE=<file>`	Parse a PlainML source file, generate output.ll, and build plainml_aot.
`make run-aot`	Run the most recently generated native executable.
`make test-aot`	Compile and execute all valid AOT regression programs.
`make test-errors`	Confirm that all invalid semantic test programs are rejected.
`make test`	Run both the valid and invalid test suites.
`make clean`	Remove all generated parser/lexer files, binaries, and output.ll.

11. Design Decisions and Current Limitations

- PlainML is intentionally inference-only; weights and biases are fixed in the source language.
- The current language accepts ReLU and step activations only.
- Connections are feed-forward and may reference only the immediately previous layer.
- A PlainML program contains one network definition followed by one prediction statement.

- Prediction values are compiled into the generated LLVM IR for the AOT executable. Runtime command-line input is not part of the language design.
- The implementation uses fixed maximum capacities: 32 inputs, 16 layers, 32 neurons per layer, and 32 connections per neuron.

12. Recommended Submission / Demonstration Flow

```
git checkout aot-compiler-dev
make clean
make test
make aot SOURCE=examples/test.plain
make run-aot
```

For a Phase 1 demonstration, switch to interpreter-dev, rebuild with make clean && make, and run ./plainml < examples/test.plain. For Phase 2, switch back to aot-compiler-dev and use the AOT commands above.

13. Troubleshooting

- "make: command not found": install GNU Make.
- "bison" or "flex" not found: install the corresponding packages.
- Linker error involving -lfl: install Flex/libfl development files (for Ubuntu, libfl-dev).
- "clang" not found: install Clang/LLVM.
- A PlainML semantic error is not a build failure; it indicates that the source program violates a language rule.
- If generated files become stale, run make clean before rebuilding.

PlainML: Flex + Bison front end, C interpreter, LLVM IR AOT backend.